

ARCADE - Documentation

Arcade is a gaming platform that implements multiple display libraries and games, allowing users to switch between them at runtime. The program is built with a modular architecture using dynamic libraries.

The project was developed as a group, with the following participants::

- paul1.emeriau@epitech.eu
- antoine.collin@epitech.eu
- alexandre.coelho@epitech.eu

The project was developed as a group, with the following participants:

- maxime.keller@epitech.eu
- thibaud.wimmer@epitech.eu

Summary:

- 1 - Architecture
 - 1.1 - Core
 - 1.2 - IDisplay
 - 1.3 - IGame
- 2 - How to compile a new library
- 3 - How to implement a graphical library
- 4 - How to implement a game

1 - Architecture

1.1 - Core

The Core is the central component of the Arcade platform. It acts as the orchestrator, managing the communication between the IGame (game logic) and IDisplay (graphical libraries). The Core is responsible for handling the game loop, processing user input, and dynamically switching between games and graphical libraries.

Responsibilities of Core:

1 - Data Transfer: Facilitates the flow of data between the game and the graphical library.

DataHolder:

The DataHolder struct is a shared container used by the Core to store and manage the current state of the application. It acts as a bridge between the Core, the game, and the graphical library, ensuring that all components have access to the necessary data. It processes events received from the graphical library. Also keep some data for the games such as the username and the total score. And indices of the currently running Game and graphical library.

```
struct DataHolder {  
    display::Event _events;           // Current event from the graphical library  
    std::string _username;           // Current player's username  
    int _score;                       // Current game score  
    bool _isTyping;                   // Indicates if the user is typing  
    int _gameIndex;                   // Index of the currently selected game  
    int _graphicIndex;                // Index of the currently selected graphical library  
    std::vector<std::string> _gameLibs; // List of available game libraries  
    std::vector<std::string> _graphiLibs; // List of available graphical libraries  
};
```

The DataHolder struct is created in the run() function and updated every loop:

```
data._events = render->getEvent(); // Capture event from IDisplay
```

The data is then sent to the game:

```
games->update(data);
```

And finally the pixels that need to be displayed are stored in a `std::vector<Pixel>` which contains only the updated pixels and are stored in `IGame` classes. This vector is then sent to the graphical library:

```
updatedPixels = games->getPixels();  
render->display(updatedPixels);
```

2 - Event Handling: Processes user input and triggers appropriate actions, such as switching libraries or updating the game state.

It also manages the game and graphical library switches through another class called `Liboader` which can take a new index and then change the running game or library. Those events are handled in those functions:

```
void eventQuit(DataHolder &data, game::IGame **games);  
void eventMenu(DataHolder &data, game::IGame **games);  
void eventLib(DataHolder &data, game::IGame **games);  
void eventGame(DataHolder &data, game::IGame **games);  
void eventRestart(DataHolder &data, game::IGame **games);
```

3 - Library Management: Dynamically loads and switches between graphical libraries and games at runtime. Here is an example of how library instances are managed:

```
display::IDisplay &getGraphicalLibInstance(const char *libPath,
void **graphHandle)
{
    void *handle;
    func_t my_entry_point;

    handle = dlopen(libPath, RTLD_LAZY);
    if (!handle) {
        fprintf(stderr, "%s\n", dlerror());
        exit(EXIT_FAILURE);
    }

    my_entry_point = (func_t) dlsym(handle, "getInstance");
    if (!my_entry_point) {
        fprintf(stderr, "%s\n", dlerror());
        dlclose(handle);
        exit(EXIT_FAILURE);
    }
    *graphHandle = handle;
    return my_entry_point();
}
```

All the tools for library management are handled through LibLoader.

```
class LibLoader
{
public:
    LibLoader(std::string grLibName="./lib/arcade_ncurses.so",
              std::string gaLibName="./lib/arcade_menu.so");
    ~LibLoader() = default;

    display::IDisplay *getGraphicLib();
    game::IGame *getGameLib();

    std::vector<std::string> getGameLibNames() const;
    std::vector<std::string> getGraphLibNames() const;

    void setGameLib(int index);
    void setGraphicLib(int index);
    std::string getLibType(const char *libPath);

private:
    void refreshNames();

    std::vector<std::string> _gameLibsNames;
    std::vector<std::string> _graphLibsNames;

    display::IDisplay *_graphicLib;
    game::IGame *_gameLib;

    void *_gameHandle;
    void *_graphicHandle;
};
```

1.2 - IDisplay:

The IDisplay interface is the backbone of the graphical library system in the Arcade platform. It standardizes rendering, input handling, and resource management, ensuring that all graphical libraries can integrate seamlessly with the Core and games. By implementing the IDisplay interface, developers can add new graphical libraries to the Arcade platform with minimal effort.

```
namespace display {  
  
    #define SDL_MULTIPLIER_X    8  
    #define SDL_MULTIPLIER_Y    9  
  
    #define SCREEN_WIDTH        240  
    #define SCREEN_HEIGHT       80  
  
    class IDisplay {  
    public :  
        virtual ~IDisplay() = default;  
  
        virtual void display(std::vector<Pixel> &pixels) = 0;  
        virtual event getEvent() = 0;  
    };  
}
```

The IDisplay interface has only two methods:

void Display():

It takes the vector of the updated pixels and has to draw them depending on the currently used graphical library.

event getEvent():

Returns the updated events that are also handled depending on the different libraries.

1.3 - IGames

IGame defines the essential structure and behavior for games, ensuring seamless integration with the Core and graphical libraries. By implementing a game using the IGame interface, we ensure seamless compatibility with the Core and any graphical libraries.

```
namespace game {  
    class IGame {  
    public :  
        virtual ~IGame() = default;  
        virtual const std::vector<display::Pixel> &getPixels() const = 0;  
        virtual void update(DataHolder &data) = 0;  
    };  
}
```

The IGame interface only has two functions:

- const std::vector<display::Pixel> &getPixels(),

Which is a getter to a vector of Pixels structure. The purpose of the function is to return the vector that is updated directly in the game.

- void update(DataHolder &data),

This function serves as the game's main loop. From here, you can implement game logic and add custom variables and handle the events that are coming from the DataHolder.

And in our own implementation we added an AGame abstract class in which we added some protected variables:

```
const std::string _name;  
std::vector<display::Pixel> _pix;  
bool _isPaused;
```

The name obviously, and a Vector of Pixel structure so that they can update it as well as a pause boolean.

The name is created in the AGame constructor and cannot be updated otherwise.

To simplify game development, we included helper functions for the pixel vector:

- void drawPixel(std::vector<display::Pixel> &pix);
- void drawRectangle(display::Position &pos, int width, int height, display::Color &color, bool fill);
- void drawCircle(display::Position &pos, int radius, display::Color &color);
- void drawText(display::Position &pos, std::string text, display::Color &color);

Those functions can be called in the game implementation and will make the life of the developer easier to draw forms or texts.

Finally we did implement a pause function which can be used in any game and is common to all the games:

- bool isPaused(DataHolder &data);

This function can be used like:

```
if (data._events.c == 'p') {  
    _isPaused = !_isPaused;  
    return;  
}  
if (isPaused(data))  
    return;
```

Calling and checking this function enables a proper pause menu and halts game updates.

2 - How to compile a new library

First, let's discuss compiling a new library. It is recommended to follow the directories so first add a new directory to the corresponding directory (GameLib or GraphLib) called MyLibraire. Then you will find a CMakeLists.txt at in the GraphLib directory, you will have to add a line to it:

- add_subdirectory(Mylibrary)

To allow Cmake to find your new graphical library. Then create the files you need in your directory. To make it compile you have to create a new CMakeLists.txt file to allow compilation.

In this example your CMake will look something like this:

```
add_library(my_library SHARED MyLibraire.cpp)

target_compile_features(my_graphical_library PRIVATE cxx_std_20)
set_target_properties(my_graphical_library PROPERTIES
POSITION_INDEPENDENT_CODE ON)

target_compile_options(my_library PRIVATE
-Wall
-Wextra
-fno-gnu-unique
)

set(LNAME "Mylibrary")

target_link_libraries(my_library // Put the dependance here)
set_target_properties(my_library PROPERTIES PREFIX "")
set_target_properties(my_library PROPERTIES OUTPUT_NAME "arcade_${LNAME}")

set_target_properties(my_library PROPERTIES LIBRARY_OUTPUT_DIRECTORY
${CMAKE_BINARY_DIR}/lib)
```


The `-fno-gnu-unique` flag is required to ensure the library is recognized by the Core. We will create some external functions which are the same for all the libraries. If this flag is not added the compiler won't allow multiple same functions.

All the instructions related to the name are to follow the arcade standard (.so files have to be named: "arcade_XXXXXX.so").

And finally don't forget to add the library as a dependance of the arcade in the CMake at the root of the project. So add the name of the library to this line:

```
DEPENDS arcade my_ncurses my_sdl2 my_sfml my_caca menu snake  
minesweeper pacman my_library
```

3 - How to implement a graphical library

For example we will call the library we are going to implement MyGraphicalLib (we will assume for the example that the events of your library are managed with integers).

Start by creating a new class in the display namespace called MyGraphicalLibraire that inherits from ADisplay and overrides the last function we need to override (display). It is highly recommended to add a catchEvent function and an enum of the corresponding events of your library to the events of the Arcade.

```
#include "Display/ADisplay.hpp"
#include <map>
#include <vector>

namespace display {
    class MyGraphicalLib : public ADisplay {
    public:
        MyGraphicalLib();
        ~MyGraphicalLib();
        void display(std::vector<Pixel> &pixels) override;
        void catchEvent();

    private:
        display::Key getKey(int ch);
        std::map<enum display::Key, int> _eventAssignment;
    };
}
```

Let's start the implementation:

First to initialize the needed values and clear them properly we will do all this in the constructor and destructor of your class to clear all the resources (don't forget to call the AGame constructor and to initialize the map of events):

```
#include "MyGraphicalLib.hpp"

namespace display {

    MyGraphicalLib::MyGraphicalLib()
    : ADisplay("MyGraphicalLib")
    {
        // Initialize your graphical library (e.g., create a window, set up colors)
        _eventAssignment = {
            {display::UP,      'w'},
            {display::DOWN,    's'},
            {display::LEFT,    'a'},
            {display::RIGHT,   'd'},
            {display::LIB,     'l'},
            {display::MENU,    'm'},
            {display::GAME,    'g'},
            {display::RESTART, 'r'},
            {display::QUIT,    27}, // Escape key
            {display::SELECT,  13}, // Enter key
        };
    }

    MyGraphicalLib::~MyGraphicalLib()
    {
        // Clean up resources (e.g., close the window, free memory)
    }
}
```

Let's implement the display function. First clear the screen using your library. If your library allows you to create a canva of pixels and add some to it, it will be better than calling draw foreach pixel and your implementation will run smoothly.

The global idea of display is:

```
void MyGraphicalLib::display(std::vector<Pixel> &pixels)
{
    // Clear the screen
    // ...

    for (const auto &pixel : pixels) {
        // Render each pixel based on its position and color
        // Example:
        // drawPixel(pixel.pos.x, pixel.pos.y, pixel.color);
    }

    // Refresh the screen to display the updated content
    // ...
}
```

We will then handle the events, for that we will implement the catchEvent function. For that use your library to get the pressed key from the user and then transform it to an arcade event using the _eventAssignment variable we created in the constructor. For that we will also add a getKey function.

```
void MyGraphicalLib::catchEvent()
{
    int ch = /* Get the pressed key using your library's input function */;

    _event = display::event(getKey(ch), ch);
}
```

```
display::Key MyGraphicalLib::getKey(int ch)
{
    for (auto [key, value] : _eventAssignment) {
        if (value == ch) {
            return key;
        }
    }
    return display::NONE;
}
```

That will fill the _event variable in ADisplay and allow the core to have access to it and then send it to the games.

The implementation is done however if you try to run the program using your library you'll get an error saying:

"ERROR: You need to specify a graphic library."

This error occurs because the Core cannot identify the library type. To fix this issue we have to create two more function to the implementation:

```
extern "C" std::string getType()
{
    return DISPLAY_TYPE;
}

extern "C" display::IDisplay &getInstance()
{
    static display::MyGraphicalLib instance;
    return instance;
}
```

DISPLAY_TYPE being a string saying: "DISPLAY". Get type will allow the Core to recognize the library and getInstance will allow the Core to get a reference to the running class and access its methods.

If you follow these instructions, your implementation should function correctly and you can now use your own graphical library to play the arcade games.

4 - How to implement a game

For example we will call the library we are going to implement MyGame.

Start by creating a new class in the game namespace called MyGame that inherits from AGame and overrides the functions that are not yet override (getPixels and update). It is highly recommended to add the pause also handleEvent and resetGame functions for your game although it is not mandatory.

```
#include "Game/AGame.hpp"
#include <vector>
#include <tuple>

namespace game {
    class MyGame : public AGame {
    public:
        MyGame();
        ~MyGame() = default;

        const std::vector<display::Pixel> &getPixels() const override;
        void update(DataHolder &data) override;

    private:
        void handleEvent(DataHolder &data);
        void drawGameElements();
        void resetGame();

        // Add any additional game-specific variables here
    };
}
```

Let's start the implementation:

In the constructor initialize all the elements you will need to run your game.

```
#include "MyGame.hpp"

namespace game {
    MyGame::MyGame() : AGame("MyGame"), _isRunning(true), _lastEvent(display::NONE)
    {
        // Initialize game-specific variables
        _isPaused = false;
        // Example: Initialize player position, score, etc.
    }
}
```

Then you will need to override the getPixels Method:

```
const std::vector<display::Pixel> &MyGame::getPixels() const
{
    return _pix;
}
```

The update method is responsible for updating the game state based on user input and game logic. It is called repeatedly by the Arcade platform. First clear the pixel vector and then add the pause options. Once this is done call your handleEvent function. You may want to draw base elements of your game before calling the handleEvent function.

```
void MyGame::update(DataHolder &data)
{
    // Clear the pixel buffer
    _pix.clear();

    // Handle pause functionality
    if (data._events.c == 'p') {
        _isPaused = !_isPaused;
        return;
    }
    if (isPaused(data))
        return;

    // Handle user input and update game state
    handleEvent(data);
}
```

In the handle event you can access the last events in `data._event` and then implement your game logic according to the events (don't forget to check the `display::RESTART` event and call your implementation of `resetGame()`).

```
void MyGame::handleEvent(DataHolder &data)
{
    if (data._events.key == display::Key::UP) {
        // Handle "up" key event
        _lastEvent = display::Key::UP;
    }
    if (data._events.key == display::Key::DOWN) {
        // Handle "down" key event
        _lastEvent = display::Key::DOWN;
    }
    if (data._events.key == display::Key::LEFT) {
        // Handle "left" key event
        _lastEvent = display::Key::LEFT;
    }
    if (data._events.key == display::Key::RIGHT) {
        // Handle "right" key event
        _lastEvent = display::Key::RIGHT;
    }
}
```

Remember to leverage the helper tools provided by AGame for streamlined implementation. We're almost done don't forget to add the `getInstance` method outside of the game namespace. Once added your game should work ! (the `getType` method is in `AGame.cpp`).

```
extern "C" game::IGame &getInstance()
{
    static game::MyGame instance;
    return instance;
}
```